

ALFACODETEAM PHPSERVICEPLATFORM · PLUGIN GUIDE

Pageflow

The Complete Tutorial — Server (PHP) & Client (React)

Pageflow is the SPA bridge for the PhpServicePlatform — a hardened fork of Inertia.js v2 wired into the kernel. Write ordinary server controllers that return a **component name + props**; the client swaps React components in place. No REST API, no client-side router, no data duplication — plus platform-native powers Inertia lacks: secure realtime, native validation & precognition, permission-aware UI, offline support, and end-to-end TypeScript types.

solves: <http://pageflow>

[@pageflow/core](#)

[@pageflow/react](#)

PHP 8.2+ · React 18

Inertia v2 wire-compatible

Generated tutorial · covers every feature & the scenario it is for

1 · What Pageflow is & how it thinks

The monolith's productivity with a modern SPA front end — and none of the API boilerplate.

In a classic SPA you build a JSON REST API, a client-side router, client-side state stores, and you duplicate validation and authorization on both sides. Pageflow deletes that layer. Your **server controllers are the API**. A controller returns a *component name* plus a bag of *props*; Pageflow decides the wire format and the client mounts (or swaps) the matching React page.

```
Browser click on <Link href="/users">
|
| XHR GET /users (header: X-Pageflow: true)
|
▼
Kernel HTTP pipeline CorrelationId → Security → Resolve → Load → Execute
|
| (+ Pageflow stages: Version, Validation, Precognition, Share)
|
▼
UserController::index() return $pageflow->render($req, 'Users/Index', ['users' => ...])
|
| 200 JSON { component, props, url, version }
|
▼
@pageflow/core router resolves "Users/Index" → swaps React component, updates history
|
|
▼ same page, no full reload — SPA feel
```

The two halves

Half	Where	Job
PHP	plugins/Pageflow/	Negotiate JSON vs HTML shell, merge shared props, validation, realtime emit, CSRF, versioning.
React	plugins/Pageflow/ui/	Router, component resolution, forms, hooks, offline, head management.

Feature map (skim this)

Need	Client	Server
Navigate	<Link>	route → controller
Read data	usePage()	render() props
Form + errors	useForm / <Form>	DTO throws ValidationException
Live validate	usePrecognition	pageflow_precognition()
Realtime	useReactiveProps	PageflowChannel::touch()
Permission UI	useAuth / <Can>	pageflow_auth (auto)
Page title	<Head>	—
Offline	registerPageflowSW()	render(..., cacheable:true)
Typed props	usePage<T>()	hkm pageflow:types

Two request shapes, one controller

- **Full page load** (first visit / refresh): server returns an HTML shell whose `<div id="app" data-page="...">` boots React.
- **Pageflow XHR** (in-app navigation): server returns the JSON page object; the client swaps the component.

Detection: the request carries `X-Pageflow` or negotiates `application/json` → JSON, otherwise HTML.

Table of contents

1 · Concepts & lifecycle

How Pageflow thinks, the two request shapes

2 · Installation & configuration

env, Provider, root view, client bootstrap

3 · Server: rendering & props

PageflowResponder, shared props, partial reloads

4 · Server: validation & precognition

throw-and-forget errors, live validation

5 · Server: realtime, CSRF, versioning

PageflowChannel, tokens, asset guards, offline

6 · Client: bootstrap & navigation

createPageflowApp, Link, router, usePage

7 · Client: forms

useForm, <Form>, usePrecognition

8 · Client: realtime, polling, prefetch

useReactiveProps, usePoll, usePrefetch, WhenVisible, Deferred

9 · Client: auth, head, state, guards

useAuth, <Can>, <Head>, useRemember, useDirtyGuard

10 · End-to-end types

hkm pageflow:types, augmenting props

11 · Scenario cookbook

CRUD, dashboard, offline, infinite scroll...

12 · Security invariants & reference

rules you must not regress, quick reference

2 · Installation & configuration

Four wiring points: environment, the Provider, the root HTML view, and the client entry.

2.1 Environment (`.env`)

```
# Bumped on every asset build – the client sends it back so a stale tab hard-reloads.
PAGEFLOW_VERSION="1"
# Absolute path to the PHP HTML shell used for full page loads.
PAGEFLOW_ROOT_VIEW="/abs/path/plugins/Pageflow/resources/views/app.php"
# The id of the root <div> the client mounts into.
PAGEFLOW_APP_ID="app"
# Session cookie the CSRF token is pinned to. MUST be in Cookie's encrypt_exempt.
PAGEFLOW_CSRF_COOKIE="hkm_session"
# Optional – MUST match the CsrfTokenLayer lifetime in withSecurity([...]).
PAGEFLOW_CSRF_LIFETIME=43200
# Optional – reactive SSE stream tuning.
PAGEFLOW_STREAM_INTERVAL=2000
PAGEFLOW_STREAM_MAX_SECONDS=300
```

CSRF binding. The token is `HMAC(APP_KEY, rawCookieValue)`. The bound cookie (`PAGEFLOW_CSRF_COOKIE`) must be listed in the Cookie plugin's `encrypt_exempt` — the `CsrfTokenLayer` reads the `raw` header before the `CookieJar` decrypts, so encryption would rotate the ciphertext and break the binding. Empty `APP_KEY` $\Rightarrow$ no token (fine for a GET-only app).

2.2 Register the plugin

Add the Provider to your project bootstrap (`projects/<name>/bootstrap/app.php`):

```
return $builder
    ->withProjectPath(dirname(__DIR__))
    ->withModules([
        Plugins\Pageflow\Provider::class,
        // ... your other plugins
    ])
    ->build();
```

Pageflow declares `requires: []`, so it is on-demand. It exposes `PageflowResponder` and `PageflowChannel`. Its two endpoints (`/pageflow/csrf`, `/pageflow/stream`) and its pipeline stages register at boot.

2.3 The root HTML view

Shipped at `resources/views/app.php`. Rendered via `ob_start()` in an **isolated scope** — only three variables exist:

`$FLOW_PAGE` (the page object), `$FLOW_CSRF` (token or `''`), `$FLOW_APP_ID`. No globals leak in.

```
<!DOCTYPE html>
<html lang="<?= $locale ?>">
<head>
    <meta charset="utf-8">
    <?php if ($FLOW_CSRF !== ''): ?>
        <meta name="csrf-token" content="<?= htmlspecialchars($FLOW_CSRF, ENT_QUOTES) ?>">
    <?php endif; ?>
    <title><?= $pageTitle ?></title>
    <link rel="stylesheet" href="/build/app.css?v=<?= rawurlencode($FLOW_PAGE->version) ?>">
    <script type="module" src="/build/app.js?v=<?= rawurlencode($FLOW_PAGE->version) ?>" defer></script>
</head>
<body>
    <!-- Emits: <div id="app" data-page="{escaped-json}"></div> -->
    <?php echo $FLOW_PAGE->mount($FLOW_APP_ID); ?>
    <noscript>This application requires JavaScript to run.</noscript>
```

```
</body>
</html>
```

The client boots from the `data-page` attribute (not `window.initialPage`). The CSRF token lives ONLY in the `<meta>` tag — never in the page-object JSON — so it never lands in an XHR body or the service-worker cache.

2.4 Client entry (`main.tsx`)

```
import { createPageflowApp, resolvePageComponent, installCsrfautoRefresh } from '@pageflow/react'
import { createRoot } from 'react-dom/client'

installCsrfautoRefresh() // auto-retry once on a 419/expired token

createPageflowApp({
  resolve: (name) => resolvePageComponent(name, import.meta.glob('./Pages/**/*.tsx')),
  setup: ({ el, App, props }) => createRoot(el).render(<App {...props} />),
  progress: { color: '#3b5bdb', showSpinner: true }, // top progress bar (false to disable)
})
```

`resolvePageComponent(name, glob)` maps a server component name like `"Users/Index"` to `./Pages/Users/Index.tsx`, awaiting lazy loaders from `import.meta.glob`. Throws a clear error if the page is missing.

3 · Server — rendering & props PHP

Everything the server does starts with one injected class: `PageflowResponder`.

3.1 The basic controller

```
use Plugins\Pageflow\Http\PageflowResponder;

final class UserController
{
    public function __construct(
        private readonly PageflowResponder $pageflow,
        private readonly UserServiceContract $users,
    ) {}

    public function index(Request $request): Response
    {
        return $this->pageflow->render($request, 'Users/Index', [
            'users' => $this->users->all(),
        ]);
    }
}
```

`render()` negotiates the wire format automatically. Signature:

```
render(
    Request $request,
    string $component,      // "Users/Index" – resolves to a React page
    array $props,           // data the page receives
    bool $loadPage = true,  // false = client supplies url/component via headers
    bool $cacheable = false // true = allow the service worker to cache this page
): Response
```

Route the controller in the plugin's `module.json` or (for a project page) in `proj.json` — never in PHP. Example route entry: `{ "method": "GET", "path": "/users", "handler": "Plugins\\Users\\Http\\UserController@index" }`.

3.2 Shared props — data on every page

Two ways. Per-request via the responder, or globally via the `pageflow_share()` helper (registered once at bootstrap, resolved per request). Page props always win on a key collision.

```
// A) In a controller / middleware – this instance only
$this->pageflow->share('flash', $flashMessage);
$this->pageflow->mergeShared(['appName' => 'HKM', 'locale' => 'en']);

// B) Global – resolved per request. Register ONCE at bootstrap or a plugin boot().
pageflow_share('year', fn() => date('Y'));
pageflow_share('cartCount', fn(Request $r) => $r->identity() ? cart_count($r) : 0);

// C) Raw contributor – full control over many keys
pageflow_share(function (Request $request, PageflowResponder $responder) {
    $responder->mergeShared(['flash' => flash_pull($request)]);
});
```

Built-in shared props you get for free: `pageflow_auth` (the identity projection for `useAuth()` / `<Can>`) and `errors` (validation errors flashed from the previous request, pull-and-clear). Both are wired in the Provider's `boot()`.

3.3 Partial reloads — send only what changed

When the client asks for a subset (`router.reload({ only: [...] })` or a `<Link only>`), the responder returns only those props. You don't change the controller — Pageflow filters the props bag based on request headers `X-Pageflow-Partial-Data` / `-Except` / `-Component` .

Optimisation: wrap expensive props in a closure so they only run when actually requested. A partial reload that excludes `stats` never pays for computing it — if you pass a closure the responder can skip it. Cheap props can be plain values.

3.4 Client-driven navigation (`loadPage: false`)

For a navigation where the client already knows the target URL/component (e.g. it rewrote history), pass `loadPage: false` ; the responder reads `X-Pageflow-Url` and `X-Pageflow-Page` to set the returned `url` / `component` . Rarely needed by hand.

3.5 The page object (wire protocol)

```
{ "component": "Users/Index",
  "props":    { "users": [ ... ], "pageflow_auth": { ... }, "errors": {} },
  "url":      "/users",
  "version":  "1",
  "clearHistory": false,
  "encryptHistory": false }
```

On a full load this same object is JSON-encoded (HTML-escaped) into the root element's `data-page` attribute by `PageflowPage::mount()` . On XHR it is the JSON body.

4 · Server — validation & precognition PHP

Throw a `ValidationException` and forget it. Pageflow turns it into the exact envelope the client's `useForm` understands — with zero manual error wiring.

4.1 The throw-and-forget flow

Your DTO validates and throws; the **PageflowValidationStage** (registered by the Provider) catches `ValidationException` and does one of three things:

Request kind	Response	Client effect
Normal Pageflow submit	Flash errors to session, 303 redirect back to origin	Origin page re-renders through the full pipeline; <code>errors</code> shared prop surfaces them
Precognition request	422 <code>{ "errors": {...} }</code>	<code>usePrecognition</code> reads it live — no side effects ran
No Session plugin	422 <code>{ "errors": {...} }</code>	<code>onError</code> handler still receives them (session recommended)
Non-Pageflow client	Re-thrown to kernel ErrorStage	Normal 422 envelope

```
final class UserController
{
    public function store(Request $request): Response
    {
        $dto = CreateUserDTO::fromRequest($request); // throws ValidationException on bad input
        $user = $this->users->register($dto);         // only reached when valid
        return $this->pageflow->render($request, 'Users/Show', ['user' => $user->toArray()]);
    }
}
```

Security: the 303 redirect target is reduced to a same-origin `/path?query` — scheme/host are stripped from the referer/ `X-Pageflow-Url` — closing an open-redirect vector.

4.2 Precognition — live server-truth validation

Precognition lets the client validate a form against the *real* server rules **without running the action** — no rule duplication in JS. A precognition request carries `Precognition: true`. Your controller must short-circuit **before any side effect**:

```
public function store(Request $request): Response
{
    $dto = CreateUserDTO::fromRequest($request); // validates (throws → 422 errors)

    if (pageflow_precognition($request)) {
        return $this->pageflow->precognitionSuccess(); // validated, no writes
    }

    $this->users->register($dto); // real work — normal submit only
    return $this->pageflow->render(/* ... */);
}
```

Helpers:

- `pageflow_precognition($request): bool` — is this a validate-only request?
- `pageflow_precognition_fields($request): array` — the subset of fields the client asked to validate (from `Precognition-Validate-Only`); empty = all. Validate just those to keep it cheap.
- `$pageflow->precognitionSuccess()` — the 2xx "valid" response.

Pair precognition endpoints with the `throttle` route filter for a hard server-side ceiling — the client debounces (300ms default) but a throttle protects against a hostile client. Optionally set `PAGEFLOW_PRECOGNITION_ROLLBACK` to auto-rollback any accidental writes in a precognition request.

5 · Server — realtime, CSRF, versioning, offline PHP

5.1 Reactive props — secure realtime in one line

The rule that makes it safe: **push signals, pull data**. The server never streams prop *values* — only the *names* of props that went stale. The client reacts with a normal authenticated partial reload, so even a forged channel can leak nothing it isn't already allowed to see.

```
use Plugins\Pageflow\Http\PageflowChannel;

final class OrderService
{
    public function __construct(
        private readonly PageflowChannel $channel, /* ... */
    ) {}

    public function place(PlaceOrderDTO $dto): OrderDTO
    {
        // ... transaction + commit ...

        // AFTER commit: mark these prop keys stale for everyone on the tenant channel.
        $this->channel->touch("dashboard:{$this->identity->tenantId}", ['orders', 'stats']);

        return OrderDTO::from($order);
    }
}
```

Channel name IS the scope. Build it from server-side identity/tenant — NEVER from client input — so one principal can't watch another's channel. Only key names travel; never data.

PageflowChannel API: `touch($channel, $keys): int` bumps a monotonic version and records stale keys in the `CachePort` (a cheap cache write, not a DB scan). `stale($channel, $since): [version, keys]` and `stream($request, $channel, $intervalMs, $maxSeconds)` power the SSE endpoint. The SSE cursor rides the `Last-Event-ID` header so a reconnecting browser resumes exactly. The stream has a bounded lifetime (default 300s) so it can't pin a worker forever.

The SSE endpoint holds a connection open — run `/pageflow/stream` under **OpenSwoole**, not a small PHP-FPM pool (each stream would occupy an FPM worker).

5.2 CSRF

The platform's `CsrfTokenLayer` expects an HMAC token in `X-CSRF-Token` on every mutating request — it is **not** the Laravel double-submit cookie scheme. The Provider mints the token per request and the root view renders it into `<meta name="csrf-token">`. The client reads that meta and echoes it back automatically.

- **Refresh endpoint** `GET /pageflow/csrf` (throttled) returns a fresh token so a long-lived tab that outlived its token can recover instead of getting a spurious 403.
- `$pageflow->csrfResponse($request)` backs that endpoint; `$pageflow->precognitionSuccess()` is the validate-only OK.

5.3 Asset versioning — force a hard reload on deploy

The **PageflowVersionStage** runs early (before modules load). When a Pageflow XHR GET carries a stale `X-Pageflow-Version`, it responds **409** with `X-Pageflow-Location`; the client does a full reload and picks up the new bundle. Bump `PAGEFLOW_VERSION` on every build (e.g. a build hash).

5.4 Offline (opt-in)

Authenticated pages are **never** cached by default. Opt a specific, user-agnostic page into the service-worker cache with `cacheable: true` — it adds `X-Pageflow-Cache: 1`:

```
return $this->pageflow->render($request, 'Marketing/Pricing', $props, cacheable: true);
```

5.5 Endpoints & config reference

Route	Purpose	Filters
GET /pageflow/csrf	Refresh CSRF token	throttle:60,1
GET /pageflow/stream	Reactive SSE channel (tenant-scoped)	auth , throttle:30,1

6 · Client — bootstrap & navigation React

6.1 `createPageflowApp`

Boots (CSR) or renders (SSR) the app. It reads the initial page from the mount element's `data-page`, resolves the initial component, decrypts history state, and calls your `setup`.

```
createPageflowApp({
  id: 'app', // must match PAGEFLOW_APP_ID
  resolve: (name) => resolvePageComponent(name, import.meta.glob('./Pages/**/*.tsx')),
  setup: ({ el, App, props }) => createRoot(el).render(<App {...props} />),
  title: (t) => `${t} · MyApp`, // title callback for <Head>
  progress: { delay: 250, color: '#3b5bdb', showSpinner: true }, // or false
})
```

SSR variant: pass `page` + `render: renderToString`; it returns `{ head, body }`.

6.2 `<Link>` — SPA navigation

```
import { Link } from '@pageflow/react'

<Link href="/users">Users</Link>
<Link href="/logout" method="post" as="button">Log out</Link>
<Link href="/users" only={['users']} preserveScroll>Refresh list</Link>
<Link href="/users" prefetch>Users (prefetch on hover)</Link>
<Link href="/users" prefetch={['mount', 'hover']} cacheFor="30s">Eager</Link>
```

Prop	Meaning
<code>href</code>	URL, or <code>{ url, method }</code>
<code>method</code>	get/post/put/patch/delete (non-GET renders a <code><button></code>)
<code>data</code>	Body/query payload merged into the URL
<code>only</code> / <code>except</code>	Partial reload — request a subset of props
<code>preserveScroll</code> / <code>preserveState</code>	Keep scroll position / component state
<code>replace</code>	Replace history entry instead of pushing
<code>prefetch</code>	<code>true</code> =hover, or <code>'mount' 'hover' 'click'</code> (array allowed)
<code>cacheFor</code>	Prefetch cache TTL (default 30s)
<code>async</code>	Non-blocking visit
<code>lifecycle</code>	<code>onBefore/onStart/onProgress/onSuccess/onError/onFinish/onCancel</code>

A `<Link>` mid-visit gets a `data-loading` attribute you can style. Normal modified clicks (ctrl/cmd/middle) fall through to the browser via `shouldIntercept`.

6.3 The `router` — imperative navigation

```
import { router } from '@pageflow/react'

router.get('/users', { page: 2 }, { preserveScroll: true })
router.post('/users', data, { onSuccess: () => toast('Saved') })
router.put('/users/1', data); router.patch(...); router.delete('/users/1')
router.reload({ only: ['stats'] }) // re-fetch some props of the current page
router.visit('/users', { method: 'get', data, only, preserveState })
router.prefetch('/users', {}, { cacheFor: 30_000 })
```

```
router.remember(state, 'key'); router.restore('key') // history-state stash
const off = router.on('start', (e) => { ... }); off() // event subscription
router.replace({ url, component, props }) // client-side page mutation
router.cancel() // router.flushAll() // router.clearHistory()
```

Events you can subscribe to: `before`, `start`, `progress`, `success`, `error`, `finish`, `navigate`, `prefetching`, `prefetched`.
Return `false` from `before` to cancel a visit.

6.4 `usePage()` — read props

```
import { usePage } from '@pageflow/react'

function UsersIndex() {
  const { props, url, component, version } = usePage<{ users: User[] }>()
  return <ul>{props.users.map(u => <li key={u.id}>{u.name}</li>)}</ul>
}
```

`usePage<T>()` is fully typed once you run `hkm pageflow:types` (§10).

7 · Client — forms React

Two APIs: the `useForm` hook (state you drive) and the `<Form>` component (declarative, serializes a native form). Both integrate errors, progress, and precognition.

7.1 `useForm` hook

```
import { useForm } from '@pageflow/react'

function CreateUser() {
  const form = useForm({ name: '', email: '', avatar: null as File | null })

  const submit = (e) => {
    e.preventDefault()
    form.post('/users', {
      preserveScroll: true,
      resetOnSuccess: ['avatar'], // or true to reset all
      onSuccess: () => toast('Created'),
    })
  }

  return (
    <form onSubmit={submit}>
      <input value={form.data.name} onChange={e => form.setData('name', e.target.value)} />
      {form.errors.name && <span className="err">{form.errors.name}</span>}
      <input type="file" onChange={e => form.setData('avatar', e.target.files[0])} />
      <button disabled={form.processing}>{form.processing ? 'Saving...' : 'Save'}</button>
      {form.recentlySuccessful && <span>Saved ✓</span>}
    </form>
  )
}
```

State	Method
<code>data</code> , <code>errors</code> , <code>hasErrors</code>	<code>setData(key, val)</code> / <code>setData(obj)</code> / <code>setData(fn)</code>
<code>processing</code> , <code>progress</code>	<code>post/put/patch/delete/get(url, opts)</code> , <code>submit(method,url,opts)</code>
<code>isDirty</code>	<code>reset(...fields)</code> , <code>resetAndClearErrors()</code>
<code>wasSuccessful</code> , <code>recentlySuccessful</code>	<code>setError</code> , <code>clearErrors</code> , <code>setDefaults</code>
—	<code>transform(fn)</code> (mutate data before send), <code>cancel()</code> , <code>replaceData()</code>

File values are sent as multipart automatically. `progress` exposes upload percent — perfect for a progress bar. Options `resetOnSuccess` / `resetOnError` take `true` or a field list.

Persisting form state across visits

```
const form = useForm('create-user', { name: '', email: '' }) // first arg = remember key
```

With a remember key, data + errors survive back/forward navigation (uses `useRemember`).

7.2 `<Form>` component — declarative

Serializes a native `<form>` (nested `name="user[email]"`, repeated `tags[]`, and File inputs all supported) and submits it. Render-prop children expose state.

```
import { Form } from '@pageflow/react'

<Form action="/users" method="post" validateOn="blur" resetOnSuccess>
  ({ errors, processing, recentlySuccessful }) => (
```

```

<>
  <input name="name" />
  {errors.name && <span>{errors.name}</span>}
  <input name="email" />
  {errors.email && <span>{errors.email}</span>}
  <button disabled={processing}>Save</button>
  {recentlySuccessful && <span>Saved ✓</span>}
</>
)}
</Form>

```

Live validation built in: `validateOn="blur"` or `"change"` (debounced, default 300ms via `validationDebounce`) fires a precognition call per field and merges the server's verdict into `errors` . Other props: `only/except` , `reset` , `transform` , `errorBag` , `preserveScroll/State` , `forceFormData` .

7.3 `usePrecognition` — manual live validation

```

import { usePrecognition } from '@pageflow/react'

const pre = usePrecognition('post', '/register', () => data, { debounce: 300 })

<input value={data.email}
  onChange={e => setData({...data, email:e.target.value})}
  onBlur={() => pre.validateField('email')} />
{pre.errors.email && <span>{pre.errors.email}</span>}
<button disabled={pre.validating} onClick={async () => {
  if (await pre.validate()) submitForReal() // whole-form gate
}}>Continue</button>

```

Returns `errors`, `hasErrors`, `validating`, `validate(fields?)`, `validateField(field)`, `clearErrors`, `setErrors` . Stale-run guarded: a newer validation never gets clobbered by a slower earlier one.

8 · Client — realtime, polling, lazy loading React

8.1 useReactiveProps — server-pushed updates

```
import { useReactiveProps } from '@pageflow/react'

function Dashboard() {
  const { props } = usePage()
  // Subscribe to the tenant channel; reload ONLY these keys when they go stale.
  useReactiveProps(['orders', 'stats'], { channel: 'dashboard', debounce: 50 })
  return <Stats data={props.stats} orders={props.orders} />
}
```

Opens an SSE stream to `/pageflow/stream`, filters incoming stale-key signals to the ones this component cares about, and debounces bursts into a single `router.reload({ only })` through the authenticated pipeline. Options: `channel`, `endpoint`, `params` (e.g. a resource id → a distinct logical channel), `debounce`, `reload`, `onError`.

Reconnection, resume-from-cursor, and credentialed same-origin auth are handled for you. If `EventSource` is unavailable (SSR/old env), it no-ops — pair with `usePoll` as a fallback.

8.2 usePoll — interval refresh

```
import { usePoll } from '@pageflow/react'

// Re-fetch just `stats` every 5s. keepAlive:false pauses when the tab is hidden.
const { start, stop } = usePoll(5000, { only: ['stats'] }, { keepAlive: false, autoStart: true })
```

8.3 usePrefetch — cache the current page

```
const { isPrefetched, isPrefetching, lastUpdatedAt, flush } = usePrefetch({ only: ['list'] })
```

Prefer `<Link prefetch>` for the common hover-to-prefetch case. Use the hook when you need to observe or invalidate the prefetch cache imperatively.

8.4 <WhenVisible> — load on scroll (infinite scroll / below-the-fold)

```
import { WhenVisible } from '@pageflow/react'

<WhenVisible data="reviews" buffer={200} fallback={<Spinner />}>
  <Reviews />
</WhenVisible>
```

Uses an `IntersectionObserver`; when the element scrolls into view (plus `buffer` px) it triggers `router.reload({ only: data })`. Pass `always` to re-fetch every time it re-enters; or `params` for custom reload options.

8.5 <Deferred> — render when a lazy prop arrives

```
<Deferred data={['permissions', 'teams']} fallback={<Skeleton />}>
  <Settings />
</Deferred>
```

Shows `fallback` until the named prop keys exist in `usePage().props`. Pair with a server that sends the page immediately and streams the expensive props via a follow-up partial reload — great for a fast first paint. It re-arms whenever a visit reloads one of its keys.

9 · Client — auth UI, head, state, guards React

9.1 `useAuth()` & `<Can>` — permission-aware UI

The server shares a non-sensitive identity projection as `pageflow_auth`. Read it for UX gating.

```
import { useAuth, Can } from '@pageflow/react'

const auth = useAuth()
auth.authenticated // bool
auth.can('invoice:create') auth.canAny('a','b') auth.canAll('a','b')
auth.hasRole('admin') auth.userId auth.tenantId auth.roles auth.permissions

<Can permission="invoice:create"><NewInvoiceButton /></Can>
<Can anyOf={['post:edit','post:moderate']} fallback=<ReadOnly />><EditBar /></Can>
<Can role="admin"><AdminMenu /></Can>
```

UX ONLY. The client can lie. `can()` / `<Can>` hide buttons; they do NOT protect data or actions. Every mutating action MUST still be authorized server-side in the Service layer. Never gate anything that matters on `can()` alone.

Customise what's shared server-side with `pageflow_auth_projection()` — e.g. send coarse capability booleans instead of your raw permission vocabulary:

```
pageflow_auth_projection(fn(?Identity $id) => [
  'userId' => $id->userId ?? '',
  'authenticated' => $id !== null && !$id->isGuest(),
  'canManage' => (bool) $id->hasPermission('admin:manage'),
]);
```

9.2 `<Head>` — document title & meta

```
import { Head } from '@pageflow/react'

<Head title="Users">
  <meta name="description" content="Manage your users" />
  <meta property="og:title" content="Users" head-key="og-title" />
</Head>
```

Deduplicates tags across page swaps via `head-key`; works in SSR (emits the head string). The app-level `title` callback (in `createPageflowApp`) can suffix every title.

9.3 `useRemember` — persist local UI state across visits

```
const [filters, setFilters] = useRemember({ q: '', sort: 'name' }, 'user-filters')
```

Survives back/forward navigation by stashing into history state (encrypted if history encryption is on).

9.4 `useDirtyGuard` — warn on unsaved changes

```
const form = useForm({ /* ... */ })
useDirtyGuard(form.isDirty, { message: 'Discard your changes?' })
```

Guards both hard browser unloads (`beforeunload`) and in-app Pageflow visits (cancels the `before` event unless confirmed). No-ops when not dirty. Set `guardUnload:false` to only guard in-app navigation.

9.5 CSRF helpers & service worker

```
import { installCsrfAutoRefresh, configureCsrf, csrfToken } from '@pageflow/react'
installCsrfAutoRefresh() // retry once on expired-token 4xx
configureCsrf({ headerName: 'X-CSRF-Token', metaName: 'csrf-token' })
const token = csrfToken() // current token or null

import { registerPageflowSW, clearPageflowSWCache } from '@pageflow/react'
registerPageflowSW({ /* scope, url */ }) // enable offline for cacheable: true pages
await clearPageflowSWCache() // e.g. on logout
```

On logout, call `clearPageflowSWCache()` and (if you swap identities) the `useFlushOnIdentityChange` helper flushes prefetch/history caches so no data bleeds across accounts.

10 · End-to-end TypeScript types

Something Inertia lacks: the CLI generates typings so `usePage().props` and component names are checked and autocompleted.

```
# Defaults: scans resources/js/Pages, writes resources/js/pageflow.d.ts
hkm pageflow:types

# Custom paths
hkm pageflow:types --pages=resources/js/Pages --out=resources/js/pageflow.d.ts
```

The generated `.d.ts`:

- Merges the always-sent shared props (`pageflow_auth`, `errors`, `csrf_token`) into `PageProps`, so `usePage().props.pageflow_auth` is typed everywhere.
- Enumerates every page under `--pages` into a `PageflowPages` registry, giving each page a typed slot and enabling component-name autocomplete.

Declare a page's own props

```
declare module '@pageflow/react' {
  interface PageflowPages {
    'Users/Index': { users: User[]; total: number }
  }
}

// then, fully typed:
const { props } = usePage<PageflowPages['Users/Index']>()
props.users      // User[]
```

Re-run `hkm pageflow:types` after adding pages or changing shared props. Wire it into your build/CI so types never drift from the server.

Testing the client

The UI ships Vitest suites (`ui/tests/`) covering CSRF, precognition, the request queue, reactive parsing, same-origin checks, URL handling and component resolution. Run with `vitest` (config at `ui/vitest.config.ts`). Mirror the platform testing pattern for controllers (fakes, never real infra).

11 · Scenario cookbook

Concrete, end-to-end recipes — pick the one that matches your problem.

Scenario A — CRUD resource (list → create → validate → redirect)

Server

```
public function index($r){ return $this->pf-
>render($r,
  'Posts/Index', ['posts'=>$this->posts->all()]); }

public function store($r){
  $dto = CreatePostDTO::fromRequest($r); // throws
  if (pageflow_precognition($r))
    return $this->pf->precognitionSuccess();
  $this->posts->create($dto);
  return Response::redirect('/posts', 303);
}
```

Client

```
const form = useForm({ title:'', body:'' })
<Form/> or:
form.post('/posts', {
  onSuccess: () => toast('Created'),
})
// errors auto-populate form.errors after the
// 303 → origin re-render → `errors` shared prop
```

When: any standard resource. **Why this shape:** the DTO is the single source of validation truth; the 303-redirect path is the secure one (errors surface through a real authorized render, not a hand-built page).

Scenario B — Live-validated signup (no rule duplication)

```
// Client: validate each field on blur against the real server rules
<Form action="/register" method="post" validateOn="blur">
  {{{errors}}} => (<
    <input name="email" />{errors.email && <i>{errors.email}</i>}
    <input name="password" type="password" />{errors.password && <i>{errors.password}</i>}
    <button>Sign up</button>
  </>)}
</Form>
// Server store(): short-circuit on pageflow_precognition($r) BEFORE writing.
```

When: registration, unique-email checks, anything where you want inline feedback but refuse to reimplement rules in JS. Add the `throttle` filter to the route.

Scenario C — Realtime dashboard (secure push)

```
// Server service, after commit:
$this->channel->touch("dashboard:{id->tenantId}", ['orders','stats']);
// Client:
useReactiveProps(['orders','stats'], { channel:'dashboard' })
```

When: live metrics, order feeds, notifications. **Why safe:** only key names are pushed; the client re-pulls through the authenticated pipeline. Run the SSE endpoint under OpenSwoole.

Scenario D — Infinite scroll / below-the-fold

```
<WhenVisible data="nextPage" buffer={300} fallback={<Spinner/>}><More/></WhenVisible>
// Server merges the extra page into props on the partial reload of `nextPage`.
```

Scenario E — Fast first paint with deferred heavy props

```
<Deferred data="analytics" fallback={<Skeleton/>}><Analytics/></Deferred>
// Send the page instantly; a follow-up router.reload({only:['analytics']}) streams the slow prop.
```

Scenario F — Offline marketing pages

```
// Server: return $this->pf->render($r, 'Marketing/Home', $props, cacheable:true);  
// Client: registerPageflowSW() // caches only X-Pageflow-Cache pages
```

Scenario G — Permission-gated toolbar

```
<Can allOf={['billing:view', 'billing:manage']} fallback={<Locked/>}>  
  <BillingActions />  
</Can>  
// Server Service still enforces billing:manage on the actual mutation.
```

Scenario H — Guard an unsaved editor

```
const form = useForm('post-editor', { title:'', body:'' })  
useDirtyGuard(form.isDirty)
```

12 · Security invariants & quick reference

12.1 Invariants you must not regress

Invariant	Why
Push signals, pull data — the reactive channel emits prop key <i>names</i> only	Data is always re-fetched through the authenticated pipeline, so a forged channel leaks nothing
CSRF token stays same-origin — lives only in the <code><meta></code> tag + throttled <code>/pageflow/csrf</code>	Never in page-object JSON or the SW cache, so it can't land in an XHR body
Client permission checks are UX only	The Service layer is the authorization boundary; the client can lie
Offline caching is opt-in (<code>cacheable: true</code>)	Authenticated pages are never cached by default
Channel name is server-built from identity/tenant	Stops one principal watching another's channel
Validation redirects are same-origin only	Scheme/host stripped from referer — closes open-redirect
Precognition short-circuits before side effects	A validate-only request must never write

12.2 Server API at a glance PHP

Symbol	Use
<code>PageflowResponder::render()</code>	Return a component + props (JSON or HTML shell)
<code>->share()</code> / <code>->mergeShared()</code>	Per-request shared props
<code>->precognitionSuccess()</code>	Validate-only OK response
<code>->csrfResponse()</code>	Backs <code>GET /pageflow/csrf</code>
<code>pageflow_share()</code>	Register global shared props (once at bootstrap)
<code>pageflow_precognition()</code> / <code>_fields()</code>	Detect validate-only request / requested fields
<code>pageflow_auth_projection()</code>	Customise the shared identity projection
<code>PageflowChannel::touch()/stale()/stream()</code>	Realtime emit + SSE
<code>hkm_pageflow:types</code>	Generate TypeScript typings

12.3 Client API at a glance React

Symbol	Use
<code>createPageflowApp</code> / <code>resolvePageComponent</code>	Boot the app / map names to components
<code><Link></code> / <code>router</code>	Declarative / imperative navigation
<code>usePage</code>	Read props/url/component/version
<code>useForm</code> / <code><Form></code>	Forms with errors, progress, precognition
<code>usePrecognition</code>	Manual live validation
<code>useReactiveProps</code> / <code>usePoll</code>	Realtime push / interval refresh
<code>usePrefetch</code> / <code><WhenVisible></code> / <code><Deferred></code>	Prefetch / scroll-load / deferred props
<code>useAuth</code> / <code><Can></code>	Permission-aware UI (UX only)
<code><Head></code>	Title & meta management

<code>useRemember</code> / <code>useDirtyGuard</code> / <code>useFlushOnIdentityChange</code>	State persistence / unsaved guard / identity flush
<code>installCsrfAutoRefresh</code> / <code>configureCsrf</code>	CSRF integration
<code>registerPageflowSW</code> / <code>clearPageflowSWCache</code>	Offline service worker

Pageflow · AlfacodeTeam PhpServicePlatform. Fork of Inertia.js v2, wire-compatible, hardened and wired into the kernel. Related docs:

`plugins/Pageflow/README.md` , `docs/ai-context/16_PLUGINS.md` , `docs/ai-context/09_SECURITY.md` , `docs/ai-context/21_CSRF.md` .